

Department of Computer and Software Engineering**SE: Machine Learning****Course Instructor:****Date:****Day:****Semester:****Lab 12: Model Deployment using Flask & Package Management**

Lab Title	CLO	BT	PLO	Weightage
Model Deployment using Flask REST APIs, Docker & uv				

Name	Roll Number	Report /10	Viva /5	Total /15

Checked on: _____

Signature: _____

Objectives

- Convert an existing API from FastAPI to Flask
- Understand differences between API frameworks
- Deploy a trained model through a web interface and containerize it
- Utilize `uv` to manage modern Python environments and resolve version conflicts

Problem Statement

In the previous lab, you built a machine learning pipeline, tracked experiments using MLflow, serialized the trained model, and deployed it using a FastAPI-based REST API.

In real-world systems, different frameworks may be used depending on project requirements. Flask is a lightweight and widely used framework for building simple web applications and APIs. Furthermore, managing dependencies for complex ML packages (like MediaPipe for computer vision) often leads to version conflicts on newer Python releases.

In this lab, you will convert the FastAPI deployment into a Flask application. Following this, you will explore environment and version management using `uv` to resolve a simulated dependency conflict.

You are required to demonstrate the working system through screenshots of:

- The running Flask application and web interfaces
- Terminal outputs demonstrating dependency errors and their resolution

Part A: Understanding Existing API

Task 1: Analyze FastAPI Implementation

Review your Lab 11 FastAPI code and identify:

- How the model is loaded
- How input is received
- How predictions are returned

Part B: Flask Setup

Task 2: Create Flask Application

Create a Flask application with:

- A home route (/)
- A prediction route (/predict)

Part C: Model Integration

Task 3: Load and Use Model

Load the serialized model from Lab 11 and use it to make predictions.

Ensure:

- Correct input format
- Proper handling of numerical features

Part D: Web Interface

Task 4: Create Input Form

Create a simple HTML form to take input features from the user.

Display prediction results on the webpage.

Part E: Testing and Screenshots

Task 5: Demonstration

Run the Flask application and capture screenshots of:

- Running server (terminal)
- Input form in browser
- Prediction output

Part F: Containerization using Docker

Task 6: Create Dockerfile

Create a Dockerfile for your Flask application.

Ensure:

- Base image uses Python
- Application dependencies are installed
- Flask app runs on port 5000

Task 7: Build and Run Docker Image

Build the Docker image and run it locally.

Verify:

- Application runs inside container
- API is accessible via browser

Screenshot Required:

- Docker build output
- Running container and browser output

Part G: AWS ECR (Elastic Container Registry)

Task 8: Create ECR Repository

Create a repository in AWS ECR.

Login to ECR using AWS CLI and authenticate Docker.

Task 9: Push Docker Image to ECR

Tag and push your Docker image to ECR.

Screenshot Required:

- Successful image push
- ECR repository showing uploaded image

Part H: Deployment using Elastic Beanstalk

Task 10: Launch Environment

Using AWS CloudShell or local CLI:

- Initialize Elastic Beanstalk application
- Configure environment (Docker platform)
- Deploy application using ECR image

Task 11: Verify Deployment

Access the deployed application via public URL.

Verify:

- Flask app is running
- Predictions are working

Screenshot Required:

- Elastic Beanstalk dashboard
- Live application URL in browser
- Prediction output

Part I: Modern Package Management with uv [Bonus]

Modern package ecosystems can be confusing: some packages publish wheels only for specific Python versions, and others require compilation that may fail on very new releases. Rather than demonstrate a single package failure (e.g., MediaPipe), this lab gives students an explicit, hands-on experience of "pain without uv" followed immediately by the solution using uv. The goal is to teach universal resolution and conditional dependencies so projects can remain installable across many Python versions.

Task 12: Feel the Pain — Installing with Plain pip

Before using uv, students will attempt a plain-pip install to experience the kinds of errors that can occur when managing Python and binary dependencies manually.

1. Create an isolated virtual environment using your system Python (do not use uv for this step):

```
python -m venv pip_test_env
# Windows (PowerShell)
pip_test_env\Scripts\Activate.ps1
# macOS / Linux
source pip_test_env/bin/activate
```

2. Attempt a plain pip install of MediaPipe:

```
pip install mediapipe
```

3. Observe and record what happens:
 - If the install fails: capture a screenshot of the terminal showing the error (build-wheel errors, missing binary wheels, or compilation failures). This is the "pain" we want students to feel.
 - If the install succeeds (note: MediaPipe began providing broader support in late April), record that it succeeded and capture the success output. Then continue — the comparison is still valuable.
4. Optional alternative (if MediaPipe installs successfully on your machine): choose a package known to require newer Python features (for example, pytensor which requires Python $\geq$ 3.10) and try installing it in a mismatched Python environment to reproduce failure modes.
5. Deactivate and remove the test environment when finished:

```
deactivate
# remove on Windows
rmdir /S /Q pip_test_env
# remove on macOS / Linux
rm -rf pip_test_env
```

6. Capture a short note (1-2 sentences) describing the experience and why manual pip installs can be fragile across Python versions and platforms.

Lab Task: Advanced Dependency Management with uv

Objective: By the end of this lab, you will understand how to manage a Python project that supports multiple Python versions. You will learn how to use environment markers to conditionally install a dependency based on the active Python version without breaking the project's universal lockfile.

Prerequisites:

- uv installed on your system.
- Basic understanding of the command line.

Step 1: Initialize the Universal Project

First, we will create a new project and explicitly declare a wide range of supported Python versions.

1. Open your terminal and create a new project named `multiversion_app`:

```
uv init multiversion_app  
  
cd multiversion_app
```

2. Open the generated `pyproject.toml` file in your code editor.
3. Locate the `requires-python` field. Change it to support Python versions greater than 3.9, up to 3.13:

```
requires-python = ">3.9, <=3.13"
```

4. Save the file. You have now promised uv that this project can run on any Python version in this range.

Step 2: The Dependency Conflict

We want to install the `pytensor` library (version 2.20.0 or higher). However, this modern library strictly requires Python 3.10 or newer.

1. Try to add the package normally:

```
uv add "pytensor>=2.20.0"
```

2. Observe the Error: uv will fail and refuse to generate a lockfile.

Question for your report: Why did uv throw this error instead of just installing the package? (Hint: Think about the promise made in Step 1).

Step 3: Resolving with Environment Markers

Instead of changing our project's minimum requirements, we will tell uv to only install `pytensor` if the user's environment is Python 3.10 or newer.

1. Add the package using a PEP 508 environment marker:


```
uv add "pytensor>=2.20.0; python_version >= '3.10'"
```

2. Observe the output. `uv` should successfully resolve the dependencies and create a `uv.lock` file.

Step 4: Simulating a Legacy Environment (Python 3.9)

Now, let's see the conditional lockfile in action by simulating a user running an older version of Python.

1. Open the `".python-version"` file in your project directory.
2. Change the contents to exactly:

```
3.9.5
```

3. Sync the virtual environment. `uv` will automatically fetch Python 3.9.5 and build the environment:

```
uv sync
```

4. Verify the installed packages:

```
uv run pip list
```

5. Observation: You should see that `pytensor` is missing from this list, but the environment built successfully without errors.

Step 5: Simulating a Modern Environment (Python 3.12)

Finally, let's switch to a newer Python version where the dependency is supported.

1. Open the `".python-version"` file again.
2. Change the contents to a modern version, such as:

```
3.12.0
```

3. Sync the virtual environment again. `uv` will tear down the old 3.9.5 environment, fetch Python 3.12.0, and rebuild:

```
uv sync
```

4. Verify the installed packages:

```
uv run pip list
```

5. Observation: You should now see `pytensor` (and its dependencies) successfully installed in the list.

Deliverables

Submit a document containing:

1. A brief explanation (1-2 sentences) of why the command in Step 2 failed.
2. A screenshot of your terminal executing `uv run pip list` during Step 4 (showing a 3.9 environment without `pytensor`).
3. A screenshot of your terminal executing `uv run pip list` during Step 5 (showing a 3.12 environment with `pytensor`).

uv Installation (Windows, macOS, Linux)

Provide students with a single canonical place to install `uv` and short, copy-paste commands for each platform. If a student cannot run the install script, they should follow the manual instructions on the official site.

Official install / docs: <https://astral.sh/uv/>

Linux / macOS (recommended):

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Windows (PowerShell):

```
powershell -ExecutionPolicy Bypass -Command "irm https://astral.sh/uv/install.ps1 | iex"
```

Notes for instructors:

- If students are uncomfortable running scripts, direct them to the official documentation at the URL above for manual installation steps and verification commands.
- After installation, verify with:

```
uv --version
```

- For macOS users preferring Homebrew, consult the documentation — the official install script covers the supported methods.

Task 13: Resolving Version Conflicts with uv

Instead of breaking or manually altering your system Python, use `uv`. It acts as a drop-in replacement for tools like `pip` and `conda`, allowing you to instantly isolate and run code with specific Python versions.

Initialize a new project with `uv` and add MediaPipe, explicitly telling `uv` to use a stable Python version (e.g., 3.11 or 3.12). Run the following commands:

```
$ uv init my-vision-project
$ cd my-vision-project
$ uv python install 3.12
$ uv add mediapipe --python 3.12
$ uv run python -c "import mediapipe; print('mediapipe successfully run')"
```

Notice how `uv` automatically handles the virtual environment and installs the correct dependencies.

Screenshot Required:

- Terminal output showing the successful execution using `uv`.

Task 15: Running Standalone Tools with uv

uv can also run Python tools that are published as packages without needing to manually install them or manage their environments.

Test this feature by running `pycowsay` directly from the command line:

```
$ uv tool run pycowsay "uv makes environment management easy!"
```

Screenshot Required:

- Terminal output showing the cow rendering your message.

Task 16: Keep Deployment Lightweight with Dev Dependencies

Add a development-only dependency (example: `pytest`):

```
$ uv add --dev pytest
```

Before deployment, sync without development dependencies:

```
$ uv sync --no-dev
```

This keeps the runtime environment lightweight by installing only production dependencies.

If dev dependencies were installed by mistake, running `uv sync --no-dev` removes them from the environment so only deploy-time packages remain.

Screenshot Required:

- Terminal output for `uv add --dev pytest`.
- Terminal output for `uv sync --no-dev`.

Task 17: Migrate requirements.txt into Modern uv Files

Create a demo `requirements.txt` file for migration practice:

```
flask>=3.0
numpy>=1.26
pandas>=2.2
scikit-learn>=1.5
```

```
requests>=2.32  
pytest>=8.0
```

If your project has an old `requirements.txt`, import it into `uv` and refresh versions:

```
$ uv add -r requirements.txt  
$ uv lock --upgrade
```

This updates project dependency management into:

- `pyproject.toml` (declared dependencies)
- `uv.lock` (resolved exact versions)

The `uv lock --upgrade` step updates packages to newer compatible releases, replacing older pinned environments with a current lockfile.

Screenshot Required:

- Terminal output for both commands.
- Updated `pyproject.toml` and `uv.lock`.

Part J: Reflection

Task 18: Reflection Questions

Answer the following:

- What is the role of Docker in deployment?
- Why use ECR instead of storing images locally?
- What are the advantages of Elastic Beanstalk?
- How does a tool like `uv` prevent "dependency hell" when working on multiple ML projects with different Python version requirements?